LangConfig —团队安装指南（中文版）

> 在全新机器上从 zip 包安装 LangConfig：PostgreSQL 安装、uv、解压、依赖同步、数据库种子（含工作区配置）、LLM API Key、健康检查与启动。

---

0. 你会收到什么

`langconfig.zip`（约 12 MB）包含完整源码，不含密钥/构建产物（无 `.env`、`node_modules`、`.venv`、`data`、`backend/outputs`、git 历史）。数据库种子会 恢复一个**与机器无关的基线**：44 个深度智能体、28 个 LLM 模型、58 个工具、19 个技能。**你的工作区路径和 API Key 在下面步骤中本地添加——zip 里不包含 任何机器相关的信息。**

---

1. PostgreSQL —安装与配置

> [!] **在启动 Postgres 之前务必先做这一步：** `docker compose` 在 compose 时会读取 `.env` 来决定宿主机端口。如果你在创建 `.env`（第 5 步）之前就启动 容器，`POSTGRES_PORT` 会回退为 `5432`，文档中的 `5433` 映射不会生效。**推荐顺序：先创建 `.env`（第 5 步），再启动 Postgres（本步骤）**——或者内联 传参：`POSTGRES_PORT=5433 docker compose up -d postgres`。

选择**一种**模式，并记下三项凭据——整个应用（后端、Alembic、种子脚本、docker-compose、SQL 智能体）都从它们派生连接信息。

项目	默认值	使用位置
数据库名	`langconfig`	`.env` 中的 `POSTGRES_DB`
用户名	`langconfig`	`.env` 中的 `POSTGRES_USER`
密码	`langconfig_dev`	`.env` 中的 `POSTGRES_PASSWORD`
主机 / 端口	`localhost`：原生 **`5432`**，Docker **`5433`**	`.env` 中的 `POSTGRES_PORT`

方案 A —Docker（最简单，团队推荐）

```
# 在解压后的目录中执行
docker compose up -d postgres
# 容器：langconfig-postgres，宿主机端口 5433（-> 容器 5432）
# 健康检查：docker ps   ->  "langconfig-postgres  0.0.0.0:5433->5432/tcp"
```

方案 B —原生 PostgreSQL 16（macOS Homebrew）

```
brew install postgresql@16
brew services start postgresql@16
# 端口 :5432；创建角色 + 数据库（或让 seed_all.py 自动创建——当应用用户
# 能以超级用户连接时它会自动建库）：
psql postgres -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev' CREATEDB;"
psql postgres -c "CREATE DATABASE langconfig OWNER langconfig;"
```

方案 C —原生 PostgreSQL 16（Ubuntu / WSL2）

```
sudo apt install -y postgresql-16 postgresql-16-pgvector   # pgvector 用于向量嵌入
sudo service postgresql start
sudo -u postgres psql -c "CREATE ROLE langconfig LOGIN PASSWORD 'langconfig_dev' CREATEDB;"
sudo -u postgres psql -c "CREATE DATABASE langconfig OWNER langconfig;"
```

角色需要 CREATEDB 权限，因为 `seed_all.py` 第 1 步会在数据库缺失时创建它，第 10 步会为 Graph Studio 创建 `chinook` 示例数据库。

---

2. 安装 uv（Python 3.13 管理器）

```
# macOS / Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
# 然后重启你的 shell 或执行:   source ~/.local/bin/env    （路径会自动添加）

uv --version          # 验证
```

需要 Python **3.13+**（仓库要求 >=3.13）。如果你的系统 Python 版本较旧，uv 可以自动拉取：

```
uv python install 3.13
```

---

3. 解压安装包

```
unzip langconfig.zip -d langconfig
cd langconfig
ls          # AGENTS.md  backend/  docker-compose.yml  docs/  package.json  src/  start.sh  s
```

---

4. 同步 Python 依赖（`uv sync`）

项目的 Python 清单是 **`backend/pyproject.toml`**（FastAPI、LangChain 1.x、LangGraph、deepagents 0.7.1、pgvector、Playwright 等）。`uv sync` 读取它并创建 `backend/.venv`。

```
cd backend
uv sync                  # 创建 .venv/ 并安装所有锁定依赖
uv run python -c "import fastapi, langgraph, deepagents; print('deps OK')"
```

> `backend/requirements.txt` 也存在，作为 pip 备选方案（`pip install -r requirements.txt`），但 **`uv sync` 是标准路径**。前端依赖是独立的：`cd .. && npm install`（仅 UI 需要；纯后端 API 不需要）。

---

5. 配置 `.env`（含工作区）——必须在种子之前

创建根目录 `.env`（唯一事实来源——后端、种子脚本、docker 和前端都读取它）。最小模板：

```
# backend/../.env   （仓库根目录）
ENVIRONMENT=development

# ── 静态加密密钥（生成一次:   openssl rand -hex 32）  ──
API_KEY_ENCRYPTION_KEY=REPLACE_WITH_64_HEX

# ── PostgreSQL（与第 1 步一致）  ──
POSTGRES_HOST=localhost
POSTGRES_PORT=5433          # docker; 原生用 5432
POSTGRES_DB=langconfig
POSTGRES_USER=langconfig
POSTGRES_PASSWORD=langconfig_dev
```

工作区参数——如何传给种子脚本

每个智能体在**本机**上都需要一个真实的工作区文件夹（它们的身份块和 AGENTS.md/CLAUDE.md 会在种子时用它重新烘焙）。两种方式：

A) 环境变量（团队推荐）：

```
export WORKSPACE_ROOT=/Users/<you>/langconfig/backend/outputs
```

智能体解析为 `<WORKSPACE_ROOT>/agents/agent_{id}/`（有自定义 `workspace_path` 的智能体使用根目录下自己的文件夹）。

B) 或使用默认值——不传时，工作区管理器自动解析为 `<repo>/backend/outputs/agents/agent_{id}/`。

---

6. 配置 LLM API Key

API Key 在运行时按以下顺序解析（`config.py::get_api_key_from_db`）：

1. **`.env` 文件**（pydantic-settings）——**权威事实来源**
2. **PostgreSQL**——`models` 设置文档中的 `api_keys` 字段(通过 Settings UI 设置,使用 `API_KEY_ENCRYPTION_KEY` 静态加密存储)

所以两种方式都可以添加 Key；如果两者都设置了，**`.env` 优先**。团队安装建议 用 Settings UI（数据库），无需编辑文件：

1. **首次启动向导**——启动后打开 `http://localhost:1421`，Setup 向导会提示 输入 API Key + 工作区路径。
2. **启动后 Settings -> Models**——添加各提供商 Key（OpenAI / Anthropic / DeepSeek / Google / MiniMax / Tavily 等）。

   你需要哪些 Key 取决于智能体使用的模型。种子智能体默认使用 Anthropic（`claude-sonnet-4-5`）/ DeepSeek 模型——至少添加与你默认模型匹配的提供商 Key，例如 `ANTHROPIC_API_KEY` 或 `DEEPSEEK_API_KEY`（`.env` **或** Settings -> Models 均可）。

---

7. 运行数据库种子

在 `backend/` 目录下执行：

```
cd backend
WORKSPACE_ROOT=/Users/<you>/langconfig/backend/outputs \
  uv run python scripts/seeds/seed_all.py
```

它会做什么（10 步，全部幂等）：

步骤	操作
1	创建 Postgres 角色 + 数据库（若缺失）
2	创建所有表 + pgvector 扩展
3	alembic stamp head
4-6	种子设置文档、28 个模型、6 个搜索提供商
7	恢复 58 个工具
8	恢复 44 个深度智能体（与机器无关的 dump）
8.5	**身份回填**——将每个智能体的真实 id + `<WORKSPACE_ROOT>/agents/agent_{id}` + AGENTS.md/CLAUDE.md 烘焙进 `system_prompt` + `agent_docs`
9	从 `backend/skills/` 同步 19 个技能
10	创建 `chinook` 示例数据库（尽力而为）

验证结果（无论你选择什么工作区根目录都能生效）：

```
uv run python -c "
from db.database import SessionLocal
from models.deep_agent import DeepAgentTemplate
db = SessionLocal()
a = db.query(DeepAgentTemplate).filter_by(id=1).first()
sp = (a.config or {}).get('system_prompt','')
print('identity:', '## Identity' in sp, '| agent_id baked:', 'Agent ID**: 1' in sp, '| no pla
db.close()
"
# 预期输出：identity: True | agent_id baked: True | no placeholder: True
```

> 之后要更改工作区,只需重新运行回填:`uv run python scripts/backfill_agent_identity.py --workspace-root < 新路径 >`

---

8. 启动前健康检查

`scripts/health_check.sh` 会在启动完整堆栈**之前**验证 Postgres、API、数据库连通性和种子数据：

```
bash scripts/health_check.sh                  # 默认后端 :8765
BACKEND_PORT=8799 bash scripts/health_check.sh   # 自定义端口
```

预期输出（全部 [OK]）：

```
── [1/4] PostgreSQL ──────────────   [OK] PostgreSQL accepting connections on :5433
── [2/4] Backend API ─────────────    [OK] Backend reachable on :8765
── [3/4] Detailed health ─────────    overall: healthy | database: healthy
── [4/4] Seeded data ─────────────    [OK] deep agents in DB: 44
[OK] ALL CHECKS PASSED — ready to run:  bash start.sh
```

如果失败，它会为每一步打印修复方法（启动 Postgres -> 种子 -> 重试）。

> 注意：该脚本需要后端**已经运行**才能检查第 2-4 步。首次启动：先启动一次后端（`bash start.sh --be`），运行检查，再执行 `bash start.sh` 启动完整堆栈。

---

9. 启动完整堆栈

```
bash start.sh
```

进程	地址	说明
后端（FastAPI，2 个 worker）	`http://localhost:8765`	Swagger 文档在 `/docs`
前端（Vite/React）	`http://localhost:1421`	VS Code 风格 IDE 外壳
PostgreSQL	`:5433`（docker）/ `:5432`（原生）	

常用变体：

```
bash start.sh --be     # 仅后端（前端保持运行）
bash start.sh --fe     # 仅前端
```

首次登录：Setup 向导会询问你的 API Key + 工作区路径——这里输入的工作区应与 第 5 步的 `WORKSPACE_ROOT` 一致，这样烘焙进智能体的身份指向运行时使用的 同一个文件夹。

故障排查

症状	修复
psycopg ... connection refused :5432/:5433	Postgres 未启动 -> 第 1 步（docker compose up -d postgres / brew services start postgresql@16）
种子第 1 步打印手动 CREATE ROLE 说明	应用用户无法以超级用户连接 -> 运行它打印的两条 psql 命令，然后重新运行 seed_all.py
健康检查后 deep agents: 0	运行第 7 步；确认 WORKSPACE_ROOT 已导出
智能体 prompt 中出现 WORKSPACE_BASE/ 或 {{...}}	回填未运行 -> uv run python scripts/backfill_agent_identity.py --workspace-root < 根目录 >
前端:1421 空白页	npm install 然后 bash start.sh --fe
API Key 不生效	在 Settings -> Models 中添加（数据库，加密）；确认 .env 中已设置 API_KEY_ENCRYPTION_KEY